

Product Requirement Document (PRD)

Project Name: RentaTool LK – Peer-to-Peer Machinery & Equipment Rental System

Module: SE3090 Software Engineering Frameworks (Year 3 | Semester 1 | 2026)

Group Size: 4 Software Engineering Members

Mandatory Stack: ASP.NET Core Web API, PostgreSQL, React, Flutter, Agentic AI

Document Status: Approved Baseline Specification

1. Project Overview & Business Objectives

1.1 Problem Statement

Across Sri Lanka, high-utility equipment (power drills, high-pressure washers, grass trimmers, rotary hammers, concrete mixers) sits idle in residential garages and small workshops for over 90% of the year. Conversely, suburban homeowners, independent trade contractors, and farmers frequently require these tools for 1–3 days. Purchasing is cost-prohibitive, and informal borrowing lacks safety guarantees, deposit protection, or dispute recourse when tools return damaged.

1.2 Solution Vision

RentaTool LK is a peer-to-peer equipment rental and security escrow ecosystem. It features:

- A **Flutter Mobile App** for renters and owners to browse local tools, generate/scan handover QR codes, upload condition photos, and manage rentals.
- A **React Web Portal** for platform operations and claim adjusters to audit identities, oversee catalog listings, monitor real-time disputes, and approve damage deductions.
- An authoritative **ASP.NET Core Web API** communicating with a **PostgreSQL** database using Entity Framework Core.
- An internal **Multi-Agent AI Subsystem** that performs automated visual and contextual damage triage, computes fair repair deductions, and pauses at high-impact transactions for human approval.

1.3 Monetization Model

- **Rental Platform Fee (Take-Rate):** 10% commission deducted from the owner's payout upon verified return.
 - **Escrow Processing Fee:** Flat fee of LKR 300 per booking covering deposit handling and automated arbitration.
 - **Commercial Store Subscriptions:** LKR 2,500/month for commercial equipment yards to receive featured ranking.
 - **Dispute Arbitration Fee:** 5% administrative surcharge on security deposit balances deducted in damage settlements.
-

2. User Personas & Role-Based Access Control (RBAC)

The system enforces three distinct user roles via ASP.NET Core JWT authentication:

- **Role: Renter (Flutter Mobile)**
 - Search & filter nearby tools using map radius.
 - Pre-authorize deposit holds and rental fees via payment gateway.

- Scan pickup/return QR codes to confirm equipment handover.
 - View active rental status and condition inspection records.
 - **Role: Owner (Flutter Mobile)**
 - List tools, specifications, replacement values, and daily rates.
 - Conduct pre-rental inspection photo uploads.
 - File damage claims with photographic evidence upon return.
 - Receive automated payouts to bank accounts upon successful return.
 - **Role: Admin / Operations Manager (React Web)**
 - Audit user KYC and National Identity Card (NIC) verification documents.
 - Catalog moderation and equipment category management.
 - Review AI-generated damage proposals and execution summaries.
 - Execute final approval, revision, or rejection of deposit deductions.
-

3. System Architecture & Boundaries

- **Backend Gateway Rule:** Both the React web portal and Flutter mobile app communicate **only** with the public ASP.NET Core Web API.
- **Internal Agent Service:** The Agentic AI subsystem (LangGraph) operates as an internal private service invoked solely by ASP.NET Core. It cannot be accessed directly by client applications.
- **Data Access & Storage:** Entity Framework Core with the PostgreSQL provider. Relational tables store transactional entities, while durable `JSONB` columns store agent execution states and audit logs.
- **Third-Party Integrations:**
 - **PayHere / Stripe Sandbox:** For card pre-authorizations,

deposit holds, and split settlements.

- **OpenStreetMap / Google Maps API:** For geolocation coordinate plotting and search radius calculations.
-

4. Work Breakdown: 4 Student Components

Each student owns one full business component end-to-end (Backend REST API, Database Entities & Migrations, React Web views, Flutter Mobile screens, Automated Tests, and a distinct AI Agent).

Component 1: User Identity, Verification & Trust Governance

Primary Owner: Student 1

- **API Endpoints:**

- `POST /api/v1/auth/register` – User onboarding with role assignment.
- `POST /api/v1/auth/login` – Issues JWT access and refresh tokens.
- `POST /api/v1/users/kyc` – Submits National Identity Card (NIC) / driving license images.
- `GET /api/v1/users/{id}/trust-score` – Computes and returns dynamic user reliability metric.
- `PATCH /api/v1/users/{id}/verification-status` – (*Business-Specific*) Admin approval/rejection of identity verification.

- **Database Entities:** `Users`, `UserRoles`, `KYCRecords`, `TrustLedger`.

- **Client Features:**

- **Flutter:** Multi-step registration, secure token storage, and camera image capture for NIC upload.
- **React:** Admin user registry with filtering, document inspection modals, and KYC approval actions.

- **Agentic AI Ownership: Validation / Safety Agent** (Validates

identity inputs, checks blacklists, and enforces access control rules).

Component 2: Equipment Catalog & Asset Condition Inspection

Primary Owner: Student 2

- **API Endpoints:**

- `POST /api/v1/equipment` – Creates a tool listing with specs, daily rate, and replacement value.
- `GET /api/v1/equipment` – Lists equipment with category filtering, search terms, and pagination.
- `POST /api/v1/equipment/{id}/inspection-logs` – Stores pre-rental/post-rental condition records.
- `GET /api/v1/equipment/{id}/history` – Retrieves maintenance and past condition timeline.
- `POST /api/v1/equipment/batch-availability` – (*Business-Specific*) Checks dynamic maintenance lockouts and flags tools requiring mandatory servicing.

- **Database Entities:** Equipment, Categories, InspectionLogs, ToolImages.

- **Client Features:**

- **Flutter:** Tool listing wizard, multi-angle camera inspection screen (casing, cord, motor) with timestamp overlays.
- **React:** Equipment inventory dashboard, category management, and condition inspection viewer.

- **Agentic AI Ownership: Domain Analysis Agent** (Classifies visual and textual condition deltas to distinguish standard wear-and-tear from structural damage).

Component 3: Booking Engine & Handover Verification

Primary Owner: Student 3

- **API Endpoints:**

- `POST /api/v1/bookings` – Initiates a booking request with start and end dates.

- GET /api/v1/bookings/active – Returns active bookings for current user.
- POST /api/v1/bookings/{id}/generate-handover-token – Issues single-use encrypted handover tokens.
- POST /api/v1/bookings/{id}/verify-handover – Confirms equipment pickup or return via QR token.
- POST /api/v1/bookings/{id}/extend-schedule – (*Business-Specific*) Validates availability conflicts and applies schedule extensions with dynamic pricing.
- **Database Entities:** Bookings, HandoverEvents, GeoLocations, BookingSchedules.
- **Client Features:**
 - **Flutter:** Interactive map radius search, QR code generator for owners, and camera-based QR scanner for renters.
 - **React:** Real-time rental tracking dashboard and schedule calendar conflict manager.
- **Agentic AI Ownership: Coordinator / Planner Agent**
(Orchestrates dispute resolution plans, sequences verification steps, and manages workflow state).

Component 4: Escrow Ledger & Security Deposit Claims

Primary Owner: Student 4

- **API Endpoints:**
 - POST /api/v1/escrow/pre-authorize – Integrates with payment gateway to lock deposit funds.
 - POST /api/v1/claims – Files a damage dispute with photo evidence.
 - GET /api/v1/claims/{id} – Fetches claim status and AI evaluation summary.
 - POST /api/v1/claims/{id}/payout – Disburses split funds to owner and refunds renter balance.
 - POST /api/v1/claims/{id}/adjudicate – (*Business-Specific*) Enforces human review decisions (Approve, Revise, Reject) and finalizes ledger settlement.
- **Database Entities:** EscrowHolds, DamageClaims, Payments, WorkflowStateAudit.

- **Client Features:**
 - **Flutter:** Escrow hold confirmation sheet, dispute filing form, and settlement summary screen.
 - **React:** Damage claim arbitration desk with side-by-side photo comparison, AI deduction proposals, and one-click approval controls.
 - **Agentic AI Ownership: Action / Tool Agent** (Executes allow-listed database and calculation tools to compute deductions based on repair pricing models).
-

5. Agentic AI Subsystem & Cross-Platform Human-in-the-Loop Workflow

5.1 The 4 Specialized Agents

1. **Coordinator / Planner Agent:** Evaluates dispute claims, loads baseline pre-rental inspection photos, and establishes a sequential verification plan.
2. **Domain Analysis Agent:** Analyzes visual differences on the tool, categorizing anomalies into normal wear-and-tear versus accidental damage requiring part replacement.
3. **Action / Tool Agent:** Calls allow-listed internal tools (`GetEquipmentReplacementCost`, `CalculateRentalWearFactor`) to compute a fair deduction amount.
4. **Validation / Safety Agent:** Validates that the calculated deduction does not exceed the escrow deposit cap, applies business rules, and prevents prompt injection.

5.2 The 6-Step Cross-Platform Loop (Assignment Mandatory Pattern)

1. **Flutter Mobile:** Owner accepts returned equipment, spots a cracked motor casing, captures a photo, and submits a damage

- dispute.
2. **ASP.NET Core:** Authenticates the request, stores initial dispute records in PostgreSQL, and invokes the internal AI service.
 3. **PostgreSQL:** Persists initial dispute state and initializes a `WorkflowStateAudit` record.
 4. **Agentic AI Subsystem:** The Planner coordinates analysis; Domain Analysis inspects photos; Action Agent calculates replacement cost (LKR 3,500); Validation Agent ensures it is within the LKR 5,000 deposit cap.
 5. **React Web (Human Approval Pause):** The workflow pauses execution. The platform administrator sees the claim marked `PendingStaffApproval` on React, inspects before/after photos, and clicks **Approve Claim**.
 6. **Shared Status:** ASP.NET Core finalizes the transaction in PostgreSQL: LKR 3,500 is disbursed to the owner, LKR 1,500 is refunded to the renter, and both users receive push notifications on Flutter.
-

6. Architecture Decision Records (ADRs)

ADR ID	Decision Area	Chosen Solution	Justification
ADR-01	React State Management	Zustand	Minimal boilerplate compared to Redux Toolkit; eliminates Context API re-rendering bottlenecks on complex data tables.
ADR-02	Flutter State Management	Riverpod / Bloc	Predictable, unidirectional data flow; decoupled UI from business logic; easily testable payment flows.

ADR-03	AI Orchestration	LangGraph	Explicit state-graph design natively supports persistent breakpoints (<code>interrupt_before</code>) required for human-in-the-loop pauses.
ADR-04	State Persistence	PostgreSQL JSONB + Relational	Relational tables guarantee financial integrity, while JSONB preserves flexible agent execution traces and tool logs.
ADR-05	Cloud Hosting	Render / Docker	Supports containerized ASP.NET Core and PostgreSQL instances on free/student tiers with zero-cost reproducibility.

7. Quality Assurance & Delivery Plan

- **Testing Spectrum:**
 - *Backend:* Unit, service layer, integration, and auth tests via **xUnit** and **Moq**.
 - *Database:* Migrations, relational constraints, and transaction rollback tests.
 - *React:* Component, routing, and form validation tests with **React Testing Library**.
 - *Flutter:* Widget, validation, and navigation tests via Flutter Test Framework.
 - *Agent Evaluation:* 20 golden dataset test cases evaluating tool selection accuracy, schema adherence, prompt injection defense, and safe-failure recovery.
- **CI/CD Pipeline:** GitHub Actions workflow executing build checks, linter runs, and unit tests on every push and pull request to `main`.
- **Deployment Deliverables:** Deployed ASP.NET Core API with

Swagger UI and health check URL; deployed React Web portal; runnable Android APK; local Docker Compose setup scripts; 10-minute demonstration video; consolidated group PDF report